

ОнкоОтвет — запуск с нуля

WDBC-демонстрация: Next.js + FastAPI + PostgreSQL + scikit-learn. Пошаговый гайд для Windows cmd.

Важно: учебная классификация benign/malignant на открытом наборе WDBC (569 записей, 30 признаков). Не прогноз эффективности лечения, не диагноз, не медицинская рекомендация. Реальные данные пациентов не использовать.

1. Что понадобится

Docker Desktop	Для варианта А. Проверка: <code>docker compose version</code> . Демон Docker должен быть запущен.
Node.js 24 LTS	Для фронта. Проверка: <code>node -v</code> и <code>npm --version</code> .
Python 3.11+	Для бэка. Проверка: <code>python --version</code> .
uv	Менеджер Python-зависимостей. Проверка: <code>uv --version</code> . Сайт: docs.astral.sh/uv .
cloudflared	Только для публичного доступа. Проверка: <code>cloudflared --version</code> . Установка: <code>winget install --id Cloudflare.cloudflared</code> .
Git (по желанию)	Проверка: <code>git --version</code> .

- Все команды ниже — для cmd (не PowerShell). Корень проекта:

C:\Users\tsay.glad\Documents\Machine Learning.

- Если в PowerShell падает `npm.ps1 UnauthorizedAccess` — это политика выполнения. Либо работайте в cmd, либо вызывайте `C:\Progra~1\nodejs\npm.cmd`.

2. Структура проекта

docker-compose.yml	Три сервиса: db (pgvector:pg17), backend (:8000), frontend (:3000). Volumes: pgdata, modeldata.
backend/	FastAPI: app/main.py (app + CORS + /health), app/core/config.py, app/db.py, app/modules/predictions/ (ml.py, schemas.py, service.py, router.py, models.py), alembic/, tests/, pyproject.toml (uv).
frontend/	Next.js App Router: src/app/page.tsx (лендинг), src/app/predictions/page.tsx (клиент, выбор образца), src/app/history/page.tsx (сервер, таблица), src/lib/api.ts.
docs/	architecture.md, implementation-plan.md и этот launch-guide.pdf.
README.md	Краткий запуск, API, проверки.

3. Вариант А — Docker Compose (рекомендуется)

3.1. Первый запуск

```
cd "C:\Users\tsay.glad\Documents\Machine Learning"
docker compose up --build
```

- Ждите, пока backend применит Alembic и поднимется API, а фронт скомпилируется.
- Web UI: <http://localhost:3000>

- API docs (Swagger): <http://localhost:8000/docs>
- Health: <http://localhost:8000/health> → {"status":"ok"}.
- Остановка: Ctrl+C, затем `docker compose down`. Данные сохраняются в volume.

3.2. Полезные команды

```
docker compose up --build &REM; первый запуск / пересборка
docker compose up -d &REM; в фоне
docker compose ps &REM; статус сервисов
docker compose logs -f backend &REM; логи API
docker compose logs -f frontend&REM; логи UI
docker compose down &REM; остановить, данные сохранить
docker compose down -v &REM; остановить И удалить БД и модель (!)
```

- Значения `POSTGRES_PASSWORD` по умолчанию — только для локальной разработки. Перед публикацией замените.
- Если поменяли `frontend/src/lib/api.ts` или `env` — делайте `up --build`, иначе возьмется старый образ.

4. Вариант Б — локально без Docker

4.1. Backend (окно cmd №1)

```
cd "C:\Users\tsay.glad\Documents\Machine Learning\backend"
uv sync
uv run alembic upgrade head
uv run uvicorn app.main:app --reload --port 8000
```

- По умолчанию SQLite backend/oncology_research.db. PostgreSQL включается через `DATABASE_URL`.
- Модель сохраняется в `backend/models/response_model.joblib` автоматически при первом запросе/старте.
- Проверка: откройте <http://localhost:8000/health> и <http://localhost:8000/docs>.

4.2. Frontend (окно cmd №2)

```
cd "C:\Users\tsay.glad\Documents\Machine Learning\frontend"
C:\Progra~1\nodejs\npm.cmd install
set API_URL=http://localhost:8000
set NEXT_PUBLIC_API_URL=http://localhost:8000
C:\Progra~1\nodejs\npm.cmd run dev -- --port 3000
```

- `API_URL` — для серверных компонентов (история). `NEXT_PUBLIC_API_URL` — для браузера (страница классификации). Локально ставьте оба на <http://localhost:8000>.
- PowerShell-вариант: `$env:API_URL="http://localhost:8000";`
`$env:NEXT_PUBLIC_API_URL="http://localhost:8000".`
- Production локально: `npm.cmd run build`, затем `npm.cmd run start -- --port 3000`.

5. Проверка API вручную

```
curl http://localhost:8000/health  
curl http://localhost:8000/api/v1/predictions/metrics  
curl http://localhost:8000/api/v1/predictions/samples
```

- /metrics вернет sample_count 569, train_fraction 0.8 и accuracy/precision/recall/roc_auc (~0.965/0.975/0.929/0.996).
- /samples вернет 4 образца с 30 признаками и эталоном benign/malignant.
- Полный цикл POST (пример для cmd с curl): возьмите features из /samples и отправьте:

```
curl -X POST "http://localhost:8000/api/v1/predictions?sample_name=Test&reference;_class=malignant" ^  
-H "Content-Type: application/json" ^  
-d '{"features": {...30 признаков из samples...}}'
```

- Успех = 201 и JSON с predicted_class, malignant_probability, model_version wdbc-logistic-regression-1.0. Неполные признаки = 422.

6. Демо-сценарий в браузере (2 минуты)

Шаг 1	Откройте http://localhost:3000 . Прочитайте дисклеймер: это не диагноз.
Шаг 2	Страница Классификация: выберите образец (видно эталон и holdout-метрики), нажмите Классифицировать образец. Сравните предсказание модели с эталоном.
Шаг 3	Страница История: таблица Дата / Образец / Эталон / Класс модели / P(malignant). Обновите после 2–3 классификаций.
Шаг 4	Откройте http://localhost:8000/docs : попробуйте GET samples/metrics и POST predictions прямо из Swagger.

7. Автопроверки (как сдать по методичке)

```
cd "C:\Users\tsay.glad\Documents\Machine Learning\backend"  
uv run pytest -q  
uv run ruff check .  
  
cd "C:\Users\tsay.glad\Documents\Machine Learning\frontend"  
C:\Progra~1\nodejs\npm.cmd run lint  
C:\Progra~1\nodejs\npx.cmd tsc --noEmit  
C:\Progra~1\nodejs\npm.cmd run build
```

- Ожидается: 7 passed, All checks passed, lint без ошибок, build со страницами /, /predictions, /history.
- Скриншоты этих команд + Swagger + UI приложите к отчету.

8. Публичный доступ через Cloudflare Tunnel (демо)

- Порядок критичен: сначала публичный бэк, потом перезапуск фронта с его URL, потом публичный фронт. Иначе браузер будет стучаться в чужой localhost.

Окно 1 — бэк с открытым CORS:

```
cd "C:\Users\tsay.glad\Documents\Machine Learning\backend"

set CORS_ORIGINS=*

uv run uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Окно 2 — туннель бэка:

```
cloudflared tunnel --url http://localhost:8000
```

- Скопируйте BACKEND_URL вида <https://xxx.trycloudflare.com>. Проверьте BACKEND_URL/health.

Окно 3 — фронт на публичный бэк:

```
cd "C:\Users\tsay.glad\Documents\Machine Learning\frontend"

set API_URL=https://xxx.trycloudflare.com

set NEXT_PUBLIC_API_URL=https://xxx.trycloudflare.com

C:\Progra~1\nodejs\npm.cmd run dev -- --port 3000
```

Окно 4 — туннель фронта (ссылка для всех):

```
cloudflared tunnel --url http://localhost:3000
```

- Раздавайте URL из окна 4. CORS_ORIGINS=* — только для демо.

9. Частые ошибки

Docker: pipe/dockerDesktopLinuxEngine not found	Не запущен Docker Desktop. Запустите Docker Desktop, дождитесь Engine, повторите up. Или используйте вариант Б без Docker.
npm.ps1 UnauthorizedAccess	Политика PowerShell. Работайте в cmd или вызывайте npm.cmd / npx.cmd полными путями.
Порт 3000/8000 занят	Найдите процесс: netstat -ano findstr :8000, завершите по PID или смените порт.
Фронт не видит API через туннель	Не задан NEXT_PUBLIC_API_URL на публичный бэк. Перезапустите фронт с обеими переменными.
CORS error в браузере	Бэк не разрешает origin фронта. Для демо: CORS_ORIGINS=*. В Compose: поправьте CORS_ORIGINS и пересоберите.
422 на POST	Переданы не все 30 признаков WDBC или лишние ключи. Берите features целиком из /samples.
Старая модель (5 признаков)	Удален автофиксом: load_model переобучается, если scaler не на 30 признаков. В крайнем случае удалите backend/models/response_model.joblib.
alembic: table already exists	База от старой схемы. Локально: удалите oncology_research.db и повторите upgrade head. В Docker: down -v (сотрет данные!).

10. Шпаргалка команд

```
docker compose up --build &REM; прод: фронт :3000, API :8000/docs, health :8000/health

docker compose down &REM; стоп с сохранением данных
```

```
cd backend & uv sync & uv run alembic upgrade head & uv run uvicorn app.main:app --reload  
cd frontend & npm.cmd install & set NEXT_PUBLIC_API_URL=http://localhost:8000 & npm.cmd run dev  
uv run pytest -q & uv run ruff check . & npm.cmd run lint & npx.cmd tsc --noEmit & npm.cmd run build
```

Подготовлено для защиты: приложите скриншоты /docs, главной, классификации, истории, зеленых тестов и docker compose ps. На слайде явно укажите: WDBC-классификация benign/malignant, не прогноз эффективности лечения.